W2026 MAD317-01 Mobile App Project Development (Kayode Olukayode)

Project Proposal, Scope, and Architecture Document

Darshilkumar Karkar
(A00203357)

StudyFlow – Mobile Study Planner Application

Platform: Android
Development Approach: Native (Kotlin)

1. Project Proposal

1.1 Project Overview

Project Title: StudyFlow – Android Study Planner

StudyFlow is an Android mobile application that is developed to assist students in organizing academic activities, assignment date, studying and individual academic objectives under one well-structured platform.

Most students use a disjointed system like notebooks, reminder tools, screenshots or memory to keep track of coursework. This usually leads to deadlines being missed, poor time management and academic stress.

StudyFlow is a solution to this issue, as it will offer:

- Centralized task tracking
- Electronic time off notifications.
- Planned time of study.
- Progress monitoring Dashboard.

The application is especially applicable in educational institutions where students have multiple courses at a given time. Time management is closely linked with high academic performance and low levels of stress.

The aim of this project is to develop and deploy a formal, safe, and performance-based study planner app to be deployed on the Android platforms.

1.2 Target Users

Primary Users

- The students of colleges and universities.
- High school students
- Students who are studying technical programs.

Secondary Users

- Adult learners
- Certification candidates
- Exam preparation students

Usage Context

- Type of Devices: Android phones (Android 10 and later)
- Connectivity: Basic features operate without the internet; optional cloud sync might need the internet.
- Environment: Campus, home, library, commute.
- Pattern of usage: Intensive, frequent sessions (2-5 minutes), every day.

User Pain Points

- Losing track of deadlines in assignments.
- Problems in prioritizing tasks.
- Lack of efficient time management.
- Inadequate planning of study.
- Feeling like overwhelmed with work.

The purpose of StudyFlow is to make life easier by displaying tasks in an articulated, orderly and easily manageable pattern.

1.3 Objectives

A. Business / User Objectives

The application will:

- Assist the students in keeping track of the work and assignments.
- Enhance time management by reminding.
- Development of academic productivity by means of planning.
- Eliminate stress through visual organization of workload.

Expected User Outcome:

Students will always submit assignments in time and enhance individual organization.

B. Technical Objectives

Performance Objectives

- Launch time of applications in the mid-range Android mobile of less than 3 seconds.
- Response time in the creation of tasks below 500 milliseconds.
- No jaggy UI and can run at 60 FPS.
- At least 500 stored tasks can be supported without any loss of performance.

Security Requirements

- Room (SQLite) Secure local storage.
- Android Biometric API optional biometric authentication.
- No unsecured transmission of data externally.
- Limited application privileges (notification and biometric access control only).

Platform Compliance

- Target Android SDK 34.
- Adhere to Material Design 3.
- Application of MVVM architecture based on Android Jetpack.
- Adhere to the privacy and data processing policies in Google Play.

Scalability Considerations

- Scalable architecture that can be later extended with cloud synergies.
- Strict division of presentation, business logic and data layers.
- Pattern of repository so that it can be expanded in the future with the API.

Every goal can be evaluated and attained in one academic semester.

2. Scope Definition

2.1 In-Scope Features

Core Features

1. User Authentication (Local)
    a. PIN/Biometric lock, which is optional.
    b. Secures the access to data on the study.
2. Task Management Module
    a. Create, edit, delete tasks.
    b. Add:
        i. Title
        ii. Description
        iii. Subject
        iv. Due date
        v. Priority level
3. Calendar View
    a. Monthly calendar display.
    b. Highlight dates with tasks.
4. Reminder System
    a. Pre-deadline local notifications.
    b. Custom reminder timing.
5. Study Session Timer
    a. In-built countdown timer (e.g., Pomodoro mode).
    b. Track completed sessions.
6. Progress Dashboard
    a. Completed vs pending tasks.
    b. Visual statistics (Simple charts).
7. Offline Data Storage
    a. Local database using Room.
8. Basic Settings
    a. Enable/disable notifications.
    b. Theme toggle (Light/Dark mode).

2.2 Out-of-Scope Features

In order to avoid scope creep, the following will not be introduced:

- Multi-user account system
- Full cloud backend server
- Online cooperation among learners.

- Artificial intelligence study recommendations.
- Cross-platform (iOS) version
- Web version
- Connection with LMS systems (e.g., Moodle, Blackboard).
- Social sharing features

This will help in ensuring the project can be completed in a single semester.

2.3 Deliverables

The project will include:

- Android APK build file
- Full source code (GitHub repository)
- Architecture documentation
- Test report (1, functional testing, and usability testing)
- Demonstration presentation
- Architectural diagram
- Requirements traceability table.

2.4 Constraints

Time Constraint

- A single academic semester (between 12-14 weeks).

Budget Constraint

- No paid APIs or services.
- Just free Android SDK and libraries.

Technical Constraint

- Development conducted by one student.
- No dedicated backend server.

Device Compatibility

- Android 10+ only.
- Coded to fit the typical smartphone screen.

2.5 Assumptions

- Android smartphones are owned by users.
- Permission to notification is enabled by users.
- Storage of devices can be made on local database.
- Multi-device synchronization is not necessary to the users.
- Google Play services are at hand (when necessary).

2.6 Risk Analysis and Mitigation

Risk	Impact	Mitigation Strategy
Notification failure	Users can have missed deadlines	Reliable background scheduling with WorkManager.
Information loss because of app crash	Loss of customer confidence	Establish correct lifecycle management and database operations.
Slowness when using a large number of tasks	Users experience slow performance	Optimize database needs and use pagination where needed.
Problems with the devices compatibility	UI inconsistency	Test across various screen sizes and Android versions
Semester time pressure	Premature feature delivery	Focus on core features and keep it within bounds.

The risk assessment will help in proactive planning and makes the projects more feasible.

3. Platform & Technology Strategy

3.1 Platform Selection Justification
Why Android?

- Larger global market share.
- Reduced development cost in comparison with iOS (no Mac needed).
- Open ecosystem.
- Appropriate to be deployed by students.

Why Native Android (Kotlin)?

- Resource management and high performance.
- Complete access to Android SDK and Jetpacks.
- Well-developed lifecycle management.
- Architectural flexibility (MVVM support) in the industry level.

It should be more integrated with device-level features like notifications and biometrics.

Trade-offs Considered

Option	Pros	Cons
Native Android	Full control, high performance	Only single platform.
Cross-platform (Flutter)	Multi-platform support	A little more complexity of architecture.
iOS only	Managing ecosystem	Hardware requirement

Native Android is chosen due to its feasibility and architectural lucidity.

3.2 SDKs, APIs, and Libraries
Development SDK

- Android SDK (API Level 34)

Programming Language

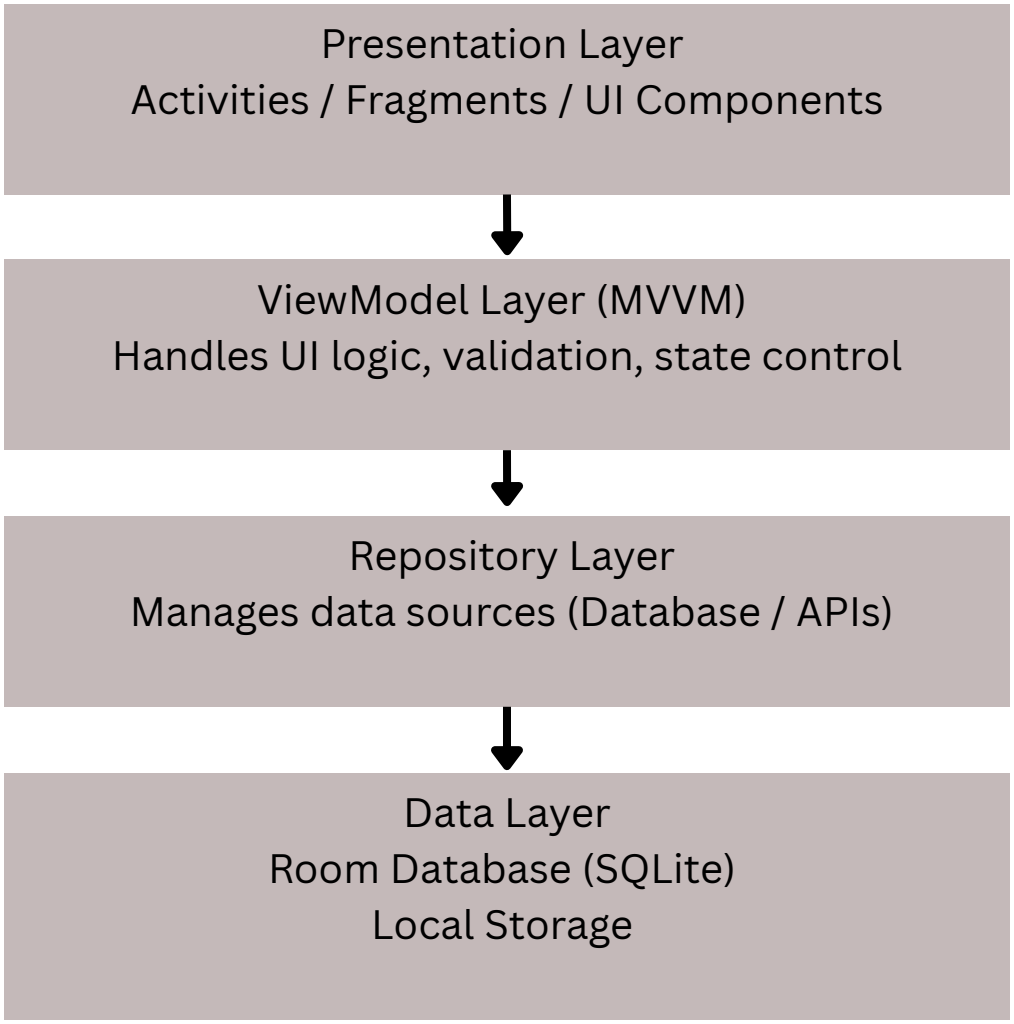
- Kotlin

Architecture Pattern

- MVVM (Model-View-ViewModel)

Tool	Purpose	Justification
Room Database	Local storage	Organized and effective
view Model	State management	Lifecycle-aware
LiveData / Flow	Data updates	Reactive UI updates
WorkManager	Scheduled tasks	Reliable background reminders
Biometric API	Authentication	Secure access
Material Components	UI design	Platform consistency

4. High-Level Architecture



External Services:

- Android Notification Manager
- Biometric Authentication API

4.1 Architecture Explanation

Data Flow

1. User interacts with UI.
2. UI makes request to ViewModel.
3. ViewModel processes logic.
4. An operation is managed in repository.
5. Information within or out of Room Database.
6. Results sent back to UI.

Security Controls

- Elective biometric verification of access to application data.
- The current implementation does not have any data transmission to the outside world.
- Information that is stored locally in form of formatted Room database.
- Minimal permission model in order to minimize attack surface.

Alignment of Design to Objectives

- MVVM ensures scalability.
- Modularity can be achieved through repository pattern.
- Room ensures performance.
- Local notification enhances usability.

4.2 Requirements Traceability

User Need	Feature	Architectural Component
Track assignments	Task Manager	Room + Repository
Avoid missed deadlines	Reminders	WorkManager
Organize workload	Dashboard	ViewModel + UI
Secure access	Authentication	Biometric API
Offline access	Local DB	Room Database

4.3 Testing Strategy

The testing of the application will be carried out in the following way:

Unit Testing

- ViewModel logic testing.
- Checking of repository layer.
- Creation and deletion validation of tasks.

UI Testing

- Fragment and Activity navigation test.
- Form validation testing.
- Reminder trigger testing.

Functional Testing

- Task notification workflow and their creation at the end.
- Verification testing- Authentication.

Usability Testing

- Test with at least 3-5 users.
- Assess the simplicity of tasks.
- Test readability of dashboard and calendar.

The testing will be done to ensure that the performance, reliability and usability goals are achieved before ultimate submission.